A

PROJECT REPORT

On

"Object Detection"

Has been

Submitted in partial fulfillment of the Project In

Diploma in Computer Engineering

By

Name of the Students	Enrollment No.	Seat No.
HARI ANJAYYA BOGA	2108540215	217075
SHIVDATT ANIL VANMANE	2208540073	217091
DHANRAJ JAYRAJ VADISHERLA	2208540087	217100
YADNESH VYANKATESH BHANDARI	2208540099	217111
PANKAJKUMAR YADAV	2208540178	217163

UNDER THE GUIDANCE OF

Prof. Biradar R.Y.

NBA Accredited & ISO 9001:2015 certified

VIDYA VIKAS PRATISHTHAN POLYTECHNIC

SOLAPUR

NBA Accredited & ISO 9001:2015 certified
VIDYA VIKAS PRATISHTHAN POLYTECHNIC,
SOLAPUR

CERTIFICATE

This is to certify that the Project entitled

"Object Detection"

Has been completed By

Of

Name of the Students	Enrollment No.	Seat No.
HARI ANJAYYA BOGA	2108540215	217075
SHIVDATT ANIL VANMANE	2208540073	217091
DHANRAJ JAYRAJ VADISHERLA	2208540087	217100
YADNESH VYANKATESH BHANDARI	2208540099	217111
PANKAJKUMAR YADAV	2208540178	217163

T.Y. (Computer Engineering)
Institute Code – 0854

Has been submitted in partial fulfillment of Project in "Diploma in Computer engineering" as per the curriculum laid by M.S.B.T.E. Mumbai, during the academic year 2024-2025

Prof. Biradar R.Y.
(Guide)

Prof. Dhobale M.R.
(H.O.D.)

(External Examiner)

(Principal)

NBA Accredited & ISO 9001:2015 certified
VIDYA VIKAS PRATISHTHAN
POLYTECHNIC,SOLAPUR

CERTIFICATE

This is to certify that MR. SHIVDATT ANIL VANMANE Seat No. 217091 of sixth semester of "Diploma in Computer Engineering" of Institute V.V.P. Polytechnic, Solapur (0854) has Completed the Final Year Project Work satisfactory in Subject "Capstone Project Execution & Report Writing (22060)" for the Academic year 2024-25 as prescribed in the curriculum.

Prof. Biradar R.Y.
(Guide)

Prof. Dhobale M.R.
(H.O.D.)

(External Examiner)

(Principal)

Declaration

We the undersigned have submitted the report for the proposed project work entitled

"Object Detection"

We declare that we have submitted the report after through study &it is not copied from any other source.

Sr.No.	Name of Students	Seat No.	Sign
1	HARI ANJAYYA BOGA	217075	
2	SHIVDATT ANIL VANMANE	217091	
3	DHANRAJ JAYRAJ VADISHERLA	217100	
4	YADNESH VYANKATESH BHANDARI	217111	
5	PANKAJ KUMAR YADAV	217163	

Allotted Guide: Prof. Biradar R.Y.

Dated Sign of Guide:

INDEX

Title	Page No
1. Chapter 1	**1-3**
Introduction	2
2. Chapter 2	**4-6**
Literature Survey	5
3. Chapter 3	**7-9**
Scope of Project	8
4. **Chapter 4**	**10-11**
Problem Definition	11
5. Chapter 5	**12-18**
Methodology	13
Data Flow Diagram	14
Use Case Diagram	16
Activity Diagram	17
Resource Required	18
6. Chapter 6	**19-26**
Code	20
Output	23
7. Chapter 7	**27-30**
Testcases	28
8. **Chapter 8**	**31-35**
Conclusion	32
Future Scope	33
References	34
Action Plan	35

ABSTRACT

This project aims to design and develop an advanced, real-time Object and QR Code Detection application using deep learning and computer vision techniques. It integrates two powerful YOLOv8 models (yolov8n.pt and yolov8n-oiv7.pt) to improve object detection accuracy across a wide range of categories. It also includes robust QR code scanning using OpenCV and Pyzbar, which extracts encoded information and redirects to web links when detected.

The GUI is built using Python's Tkinter and CustomTkinter libraries, offering a simple, responsive, and interactive user interface. The application provides multiple modes: real-time webcam detection, QR scanning, and image-based object detection from the gallery. Users can easily switch between modes, with support for dark/light themes, a splash screen, and live detection previews.

Main features include bounding boxes with confidence scores, dual-model inference, and automatic handling of QR code links. This system is useful in areas such as smart surveillance, warehouse management, educational applications, and access control.

Overall, this project demonstrates efficient AI model integration and real-time visual processing through a user-friendly interface.

RATIONALE

Object detection is a necessary technology because it addresses critical real-world needs across various industries and everyday life. Implementing object detection enhances safety, improves efficiency, and reduces human error. For instance, in security, it allows for real-time monitoring of activities. In industries like retail and manufacturing, it automates inventory management and quality control, saving time and resources. Object detection also plays a vital role in assisting visually impaired individuals by helping them recognize objects in their surroundings..

CHAPTER 1
INTRODUCTION

INTRODUCTION

In recent years, the fusion of artificial intelligence and computer vision has opened up numerous possibilities for real-time image processing and automation. Among these, object detection and QR code scanning have emerged as crucial technologies across industries such as retail, security, logistics, smart cities, and automation. This project aims to design and implement an interactive application that combines both object detection and QR code recognition using powerful deep learning models and an intuitive graphical user interface (GUI).

The core functionality of the system is built upon the YOLO (You Only Look Once) model, specifically the YOLOv8 and YOLOv8-OIV7 variants, known for their high-speed and accurate object detection capabilities. These models are pre-trained on large-scale datasets and are capable of detecting multiple objects in real-time. The application uses two models simultaneously to increase detection robustness and provide comparison insights. For decoding QR codes, the application utilizes the pyzbar library in combination with OpenCV, a widely-used computer vision library, to recognize and interpret QR codes from images and live camera feeds.

This project includes multiple operation modes such as real-time object detection via webcam, QR code scanning in real-time, and gallery-based detection from static images. Users can interact with the system through a simple GUI created using Python's Tkinter library. The interface is designed to provide ease of use with dedicated buttons for each detection mode, and results are visually displayed with bounding boxes and labels for identified objects or scanned QR code data.

Moreover, the application introduces user-centric features such as real-time redirection to URLs encoded in QR codes, error handling for unsupported image formats, and an enhanced viewing window for displaying detected images. An extended version of the app also uses CustomTkinter to enable modern UI aesthetics including a dark/light mode toggle and a splash screen, making the system more visually appealing and professional.

One of the notable aspects of this system is its standalone capability. All models and dependencies are bundled in a way that allows the application to run without requiring extensive configuration or internet access. This makes the tool especially useful for field environments or offline use cases.

From a development perspective, the project also emphasizes modular programming, code reusability, and clean interface design. Each function such as image loading, QR decoding, object detection, and GUI rendering is encapsulated within well-defined blocks of code.

This modularity not only enhances readability and maintainability but also allows for easy scaling or upgrading of features in the future.

The practical applications of such a system are extensive. In retail, it could be used to track inventory and read product codes. In logistics, it could aid in package scanning and sorting. In security, it could assist with surveillance and monitoring tasks. In educational setups, it can serve as a real-time demonstration of AI-powered vision systems.

In conclusion, this project serves as a comprehensive implementation of a multi-modal visual detection tool that harnesses the power of deep learning, image processing, and user interface design. It offers a strong foundation for future enhancements such as cloud-based facial recognition, vehicle number plate verification, and multilingual text extraction. By combining state-of-the-art AI models with user-friendly interaction, this application stands as a useful tool for both technical experimentation and real-world deployment.

CHAPTER 2

LITERATURE SURVEY

LITERATURE SURVEY

1.Real Time Object Detection using TensorFlow

Authors - Aakanksha Singh, Madhavi Soni, Shreya Patkar, Asmita Deshmukh.

Authors have implemented mobile application to detect real time object. System uses camera to capture object and displays the name of the object on the screen in real time. Sometimes people don't recognize any new object and don't know that what it is. With the help of this application uses can easily recognize about unknown objects. This system uses Tensor Flow library for the Object Detection. This also reduces the human efforts and saves time.

This tool is especially useful when facing unfamiliar objects, as it provides immediate identification, as we search for anything on internet instead of that we can detect the object by this app and know what it is. This app can be used in everyday situations where we need quick recognition and the main motive for developing an Object Detection App is that can automatically identify objects in real time using a camera. We will be training the system by adding various dataset.

2. Object Detection Using OPEN CV, TensorFlow and Flask

Authors – Hemalatha T., K Venkata Chalapathy

Object Detection is process of finding and recognizing real time object and image's and video's, by this technique you understand information about captured real-time/image scanned object by this process human can easily identify number of objects in image or video are present the human visual system cannot visual multiple objects at same time many objects so this process will generate result with high accuracy available objects in real-time/captured image

Technology used:

OPEN CV - Open CV is a cross-platform library using which we can develop real-time computer vision applications, it processes images and videos to identify objects, faces and text/human handwritings

3.Real Object Detection Using TensorFlow

Authors - Milind Rane, A seem Patil and Bhushan Barse

In this project, author used tensor flow to develop object detection application which presents a new method. They used YOLO technique for real time image processing, which is faster version with a smaller architecture of 155 frames per sec, but is less accurate.

For camera, Open CV is used, which is cross platform library using this we can develop real-time computer vision application, it process images and videos to identify objects, faces, etc.

4. Object Detection with OpenCV using Python

Authors : Agnes , Sam salmon raja , Pushparaj ,Abrar A.

This paper presents a novel approach to enhancing accessibility for visually impaired individuals through object detection technology implemented using OpenCV. Leveraging state-of-the-art object detection models, we develop a real-time system capable of detecting objects in camera feeds and providing auditory feedback for navigation and interaction with the environment.

The integration of OpenCV enables efficient object detection, bounding box visualization, confidence thresholding, and non-maximum suppression, contributing to the development of assistive technologies for the visually impaired. Through a comprehensive review of existing literature, we identify gaps and opportunities for innovation in developing accessible solutions for the visually impaired

5. Real Object Detection Using TensorFlow

Authors : Rajeshwar Kumar Dewangan1, Yamini Chouhan2

Object detection using OpenCV is an important part of computer vision, which focuses on recognizing and locating objects in images or videos. There are two main ways to do this: traditional methods and modern deep learning techniques.

Traditional Methods: These methods rely on simple, handcrafted features, meaning that certain patterns like edges, colours, or shapes are manually defined. These features help the system recognize objects. Some common examples include:

Contour Detection: This method finds the outlines of objects by detecting differences in brightness or colour in an image, helping to identify object boundaries.

CHAPTER 3
SCOPE OF PROJECT

Object Detection using OpenCV : Scope

This project aims to develop a comprehensive application that integrates object detection and QR code scanning functionalities. The application leverages advanced machine learning models, specifically YOLOv8, for object detection and the PyZbar library for QR code recognition. The user interface is built using Tkinter, providing a seamless experience for users to interact with the system. The application supports real-time detection, image-based detection, and QR code scanning from both live camera feeds and gallery images.

Objectives

1. **Object Detection**: Utilize YOLOv8 models to detect and classify objects within images and live video streams. The application will support multiple models to enhance detection accuracy and versatility.

2. **QR Code Scanning**: Implement QR code detection and decoding using the PyZbar library. The system will highlight detected QR codes and provide options to open URLs embedded within the codes.

3. **User Interface**: Develop an intuitive GUI using Tkinter to facilitate easy navigation and interaction. The interface will include options for real-time detection, image uploads, and QR code scanning.

4. **Real-Time Processing**: Enable real-time object detection and QR code scanning using live camera feeds. The system will display the processed video stream with detected objects and QR codes highlighted.

5. **Image Processing**: Allow users to upload images for offline object detection and QR code scanning. The system will display the processed images with detection results.

Features

1. **Real-Time Object Detection**:

- Detect and classify objects in real-time using a webcam feed.
- Display bounding boxes and labels for detected objects.

2. **Real-Time QR Code Scanning**:

- Detect and decode QR codes in real-time.
- Highlight QR codes and provide options to open embedded URLs.

3. **Image-Based Object Detection**:

- Upload images for object detection.
- Display processed images with detected objects and their labels.

4. **QR Code Scanning from Gallery**:

- Upload images containing QR codes for offline scanning.
- Display processed images with detected QR codes and decoded information.

5. **User-Friendly Interface**:

- Simple and intuitive GUI for easy navigation.
- Buttons for different functionalities (real-time detection, image uploads, QR code scanning).

CHAPTER 4
PROBLEM DEFINITION

PROBLEM DEFINITION

Develop an application using vision technology to replicate the ability of To recognize and locate objects in real-time.

CHAPTER 5
METHODOLOGY

METHODOLOGY

Object detection is a fundamental aspect of computer vision that involves identifying objects within images or videos. In our project, we used OpenCV along with YOLOv8 (You Only Look Once, version 8), a state-of-the-art deep learning model, to implement efficient and accurate object detection. The process follows a structured workflow:

1. **Capturing Input**: The process begins with capturing an image or video, which serves as the input for the system. This input can be obtained from a camera, a stored file, or a video stream.
2. **Preprocessing**: Before analysis, the captured input undergoes preprocessing. This step includes resizing the image to a suitable dimension, applying filters to enhance quality, and converting it into a format that the YOLOv8 model can efficiently process. Preprocessing ensures that unnecessary noise is reduced and relevant features are preserved.
3. **Object Detection Using YOLOv8**: We applied YOLOv8 for object detection, which uses a deep neural network to analyze the image and identify objects with high accuracy and speed. YOLOv8 processes the entire image in one pass, making it highly efficient for real-time applications. It provides bounding boxes and class labels for detected objects, allowing precise identification.
4. **Post-Processing**: Once objects are detected, the system highlights them using bounding boxes and assigns labels with confidence scores. Post-processing ensures that detected objects are clearly marked for further analysis or decision-making. If needed, non-maximum suppression (NMS) is applied to remove overlapping detections and improve accuracy.
5. **Displaying Results**: The final detected objects, along with their corresponding labels and confidence scores, are presented to the user. The results can be displayed on a screen, stored for further processing, or integrated into an application for automation and real-time decision-making. Our implementation provides a user-friendly interface for interacting with the detected results.

By using YOLOv8 with OpenCV, our project achieves high-speed and accurate object detection, making it suitable for various real-world applications such as security surveillance, autonomous vehicles, medical imaging, industrial automation, and augmented reality. YOLOv8's advanced deep learning capabilities ensure reliable detection even in complex environments, further enhancing its practical applications.

DATA FLOW DIAGRAM

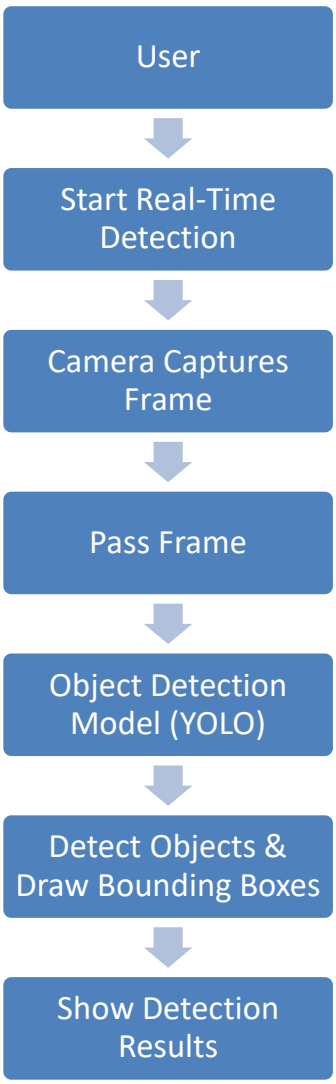


Fig 1. Dataflow Diagram: Real Time Object Detection

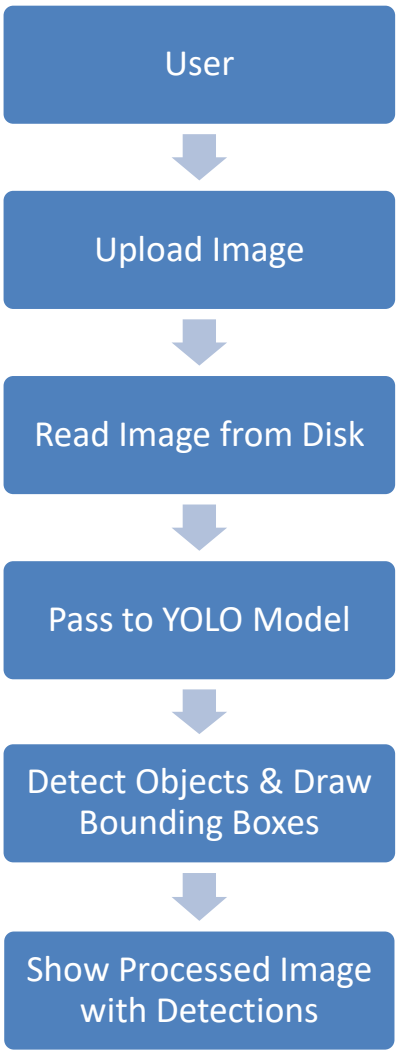


Fig 2. Dataflow Diagram: Image Detection

USE CASE DIAGRAM

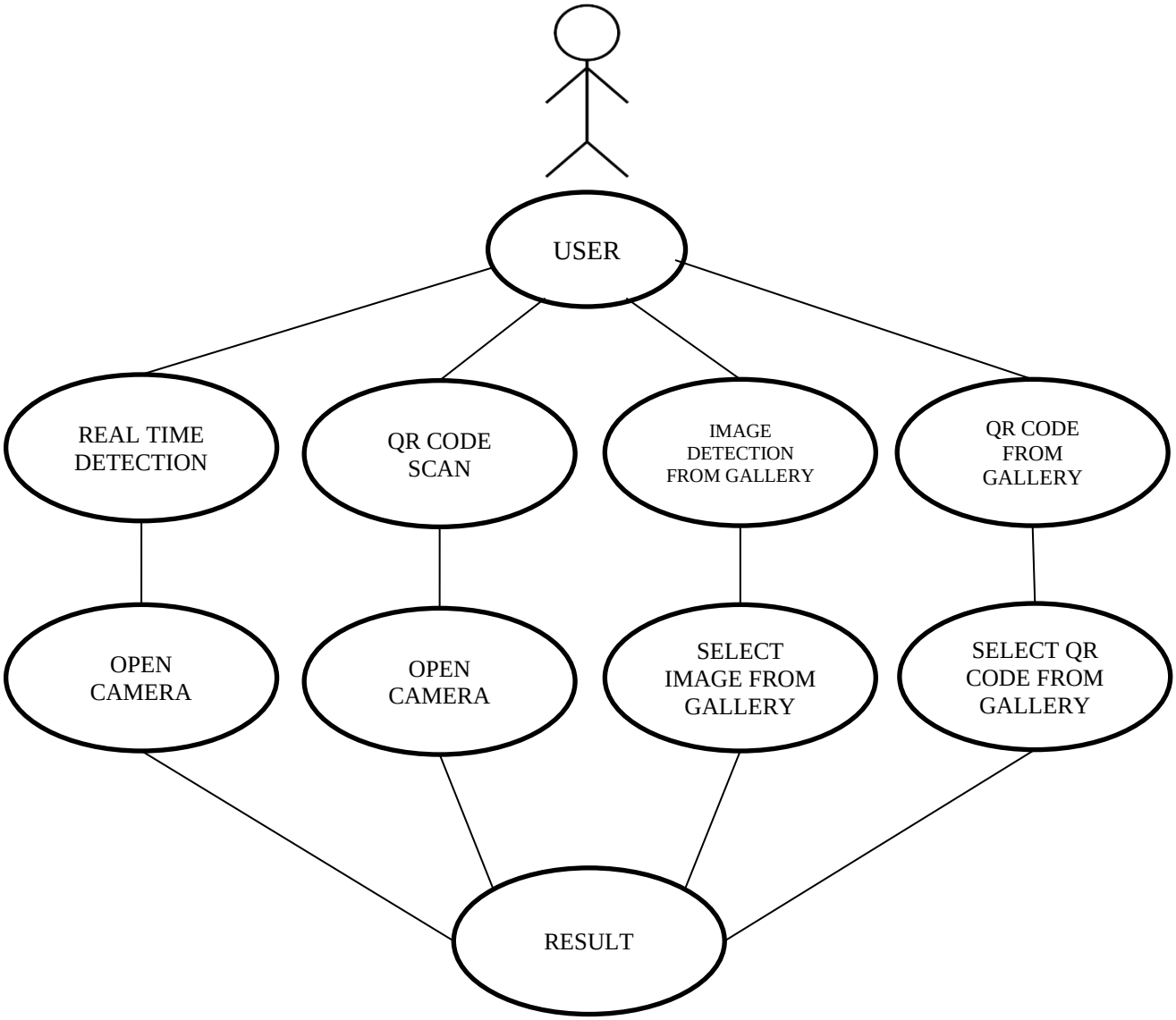


Fig 3. Dataflow Diagram: Image Detection

ACTIVITY DIAGRAM

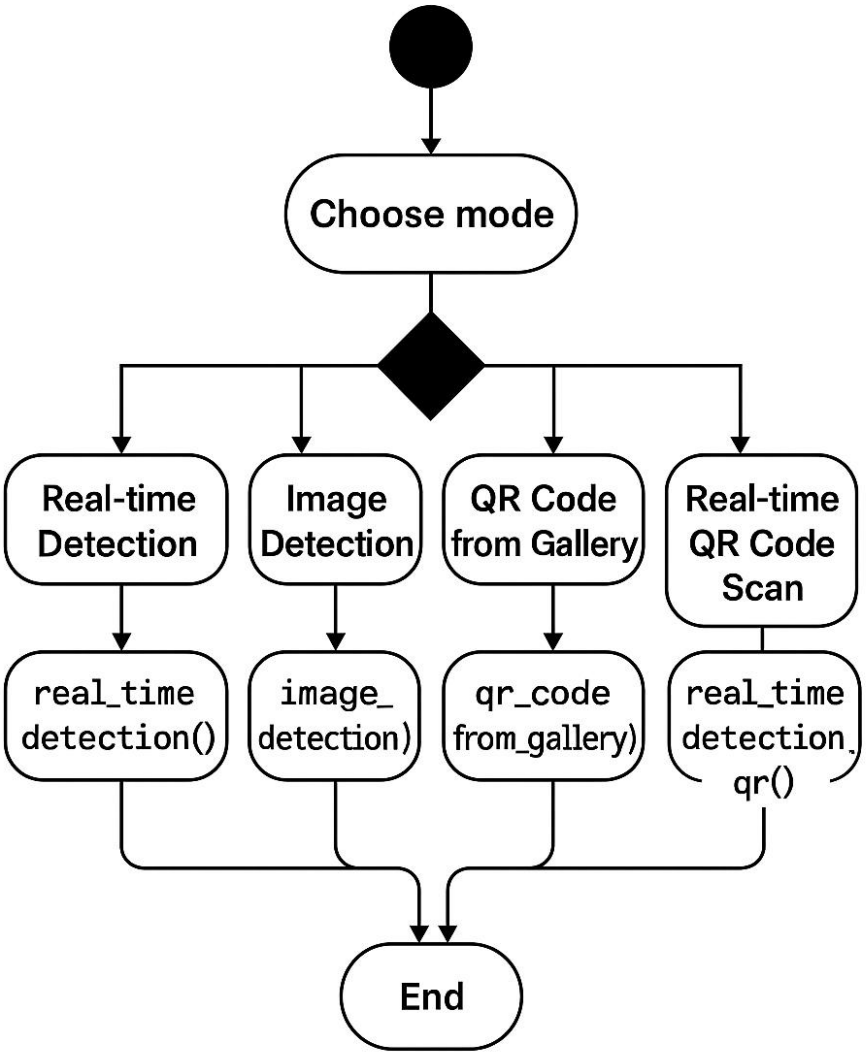


Fig 4. Dataflow Diagram: Image Detection

RESOURCES & CONSUMABLES REQUIRED

Minimum HARDWARE REQUIREMENTS

- Hard Disk: 256 GB.
- Ram: 4.00 GB
- Monitor: 15 VGA Color
- Mouse: Logitech.

Minimum SOFTWARE REQUIRMENTS

- Operating system: Windows 10
- Language: Python
- Libraries: OpenCv,
- IDE: Visual studio Code

CHAPTER 6
CODE & OUTPUT

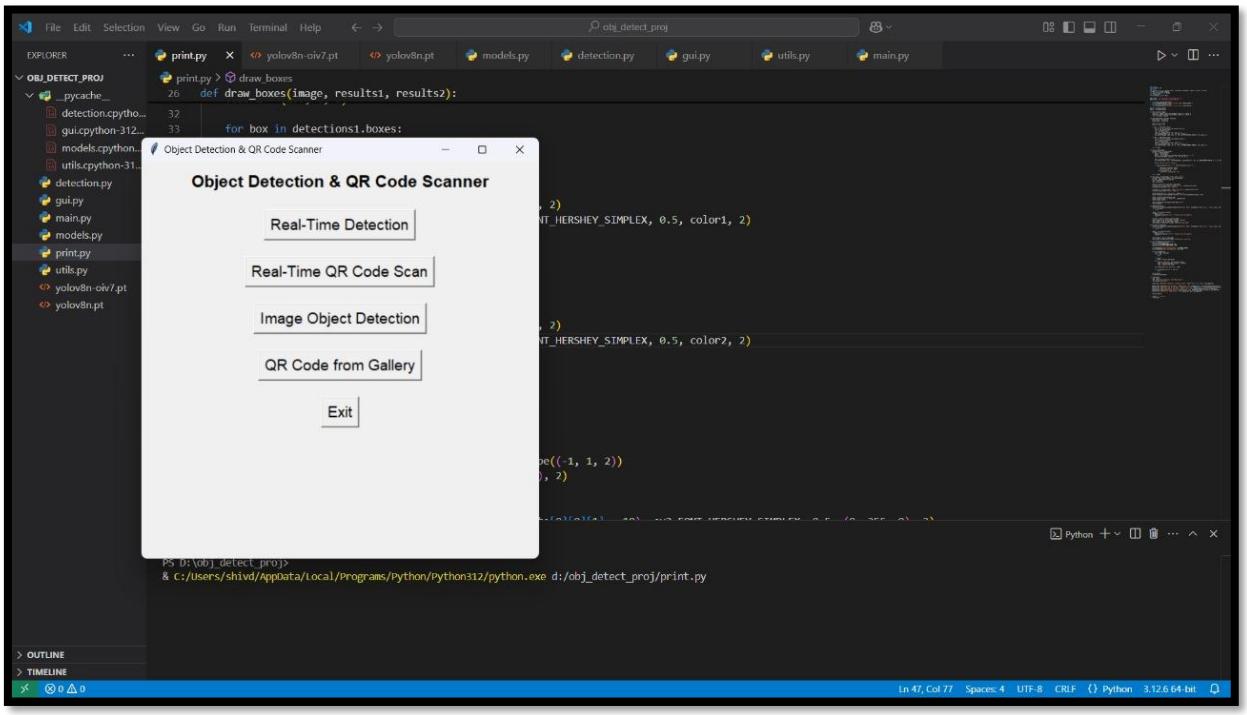
Code :

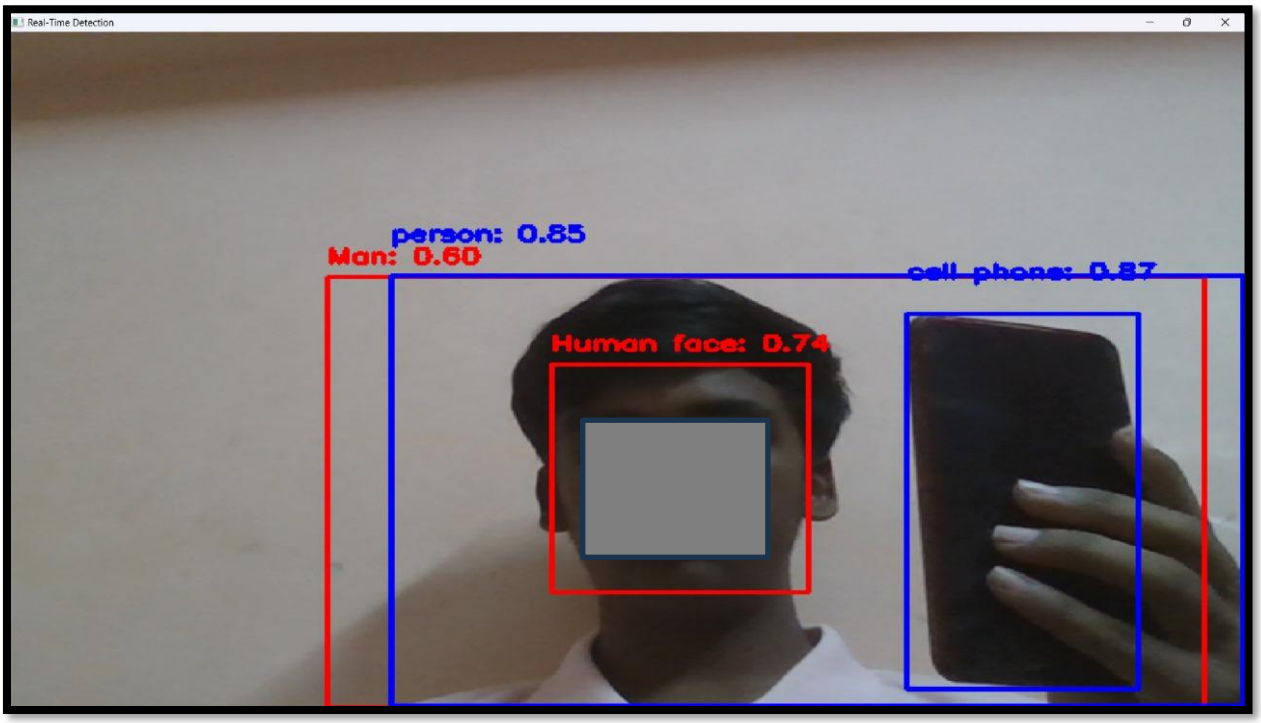
```
import cv2

import numpy as np

import os

from tkinter import Tk, Button, Label, filedialog, messagebox, Toplevel, Canvas, Scrollbar

from PIL import Image, ImageTk

from pyzbar.pyzbar import decode

import webbrowser

from ultralytics import YOLO

model_path1 = r"D:\obj_detect_proj\yolov8n-oiv7.pt"

model_path2 = r"D:\obj_detect_proj\yolov8n.pt"

if not os.path.exists(model_path1):

    raise FileNotFoundError(f"Model file not found: {model_path1}")

if not os.path.exists(model_path2):

    raise FileNotFoundError(f"Model file not found: {model_path2}")

model1 = YOLO(model_path1)

model2 = YOLO(model_path2

def detect_objects(image):

    results1 = model1.predict(source=image, save=False, conf=0.5)

    results2 = model2.predict(source=image, save=False, conf=0.5)

    return results1, results2

def draw_boxes(image, results1, results2):

    detections1 = results1[0]

    detections2 = results2[0]

    color1 = (0, 0, 255)

    color2 = (255, 0, 0)

    for box in detections1.boxes:

        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())

        conf = box.conf[0].item()

        cls = int(box.cls[0])

        label = f"{model1.names[cls]}: {conf:.2f}"

        cv2.rectangle(image, (x1, y1), (x2, y2), color1, 2)

        cv2.putText(image, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color1, 2)

    for box in detections2.boxes:

        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())

        conf = box.conf[0].item()

        cls = int(box.cls[0])

        label = f"{model2.names[cls]}: {conf:.2f}"

        cv2.rectangle(image, (x1, y1), (x2, y2), color2, 2)

        cv2.putText(image, label, (x1, y1 - 25), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color2, 2)

    return image

def scan_qr_code(image):

    decoded_objects = decode(image)

    for obj in decoded_objects:

        points = obj.polygon

        points = np.array(points, dtype=np.int32).reshape((-1, 1, 2))

        cv2.polylines(image, [points], True, (0, 255, 0), 2)
```

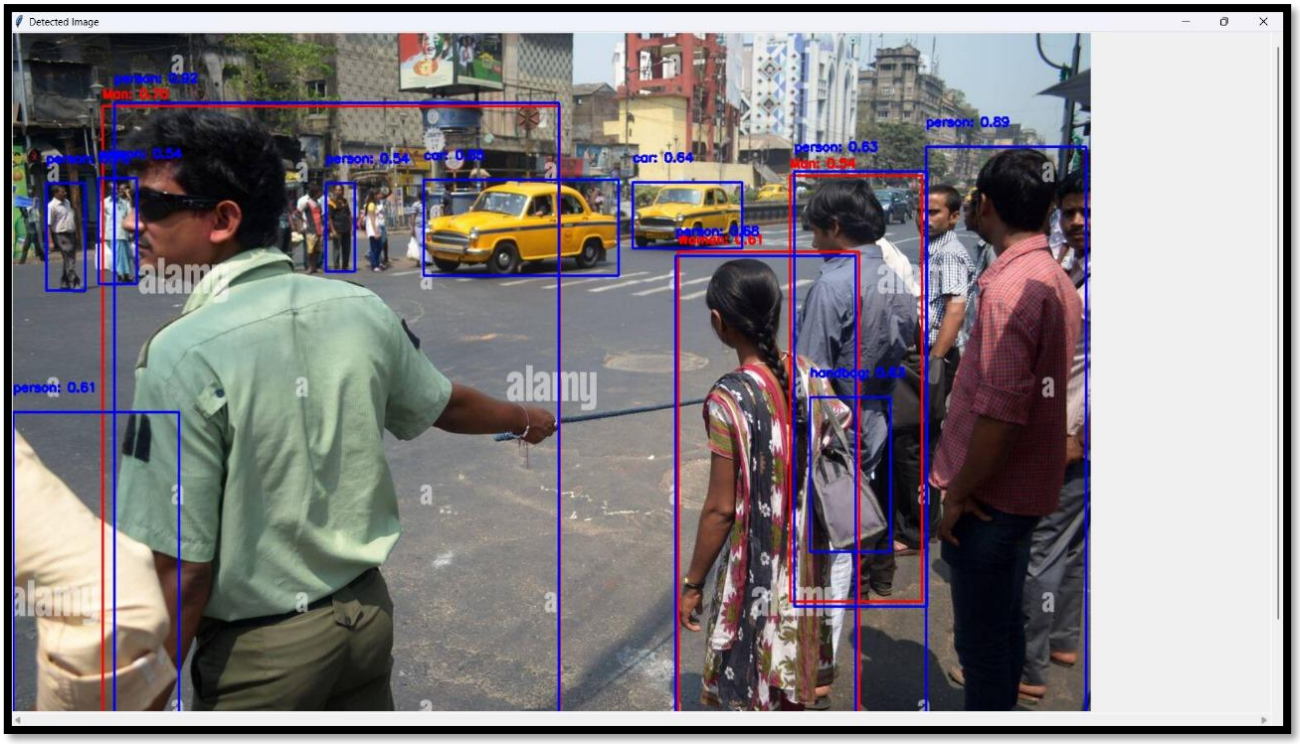
```
        text = obj.data.decode("utf-8")

        cv2.putText(image, text, (points[0][0][0],
points[0][0][1] - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,
255, 0), 2)

        print(f"QR Code detected: {text}")

        if text.startswith("http://") or
text.startswith("https://"):

            try:

                webbrowser.open(text, new=2)

                print("Opening URL:", text)

            except Exception as e:

                print("Error opening URL:", e)

    return image

def show_image_in_window(image,
title="Image Viewer"):

    img_rgb = cv2.cvtColor(image,
cv2.COLOR_BGR2RGB)

    pil_img = Image.fromarray(img_rgb)

    root = Toplevel()

    root.title(title)

    canvas = Canvas(root, width=800,
height=600)

    scrollbar_y = Scrollbar(root,
orient="vertical", command=canvas.yview)

    scrollbar_y.pack(side="right", fill="y")

    scrollbar_x = Scrollbar(root,
orient="horizontal", command=canvas.xview)

    scrollbar_x.pack(side="bottom", fill="x")


    canvas.pack(side="left", fill="both",
expand=True)


canvas.configure(yscrollcommand=scrollbar_
y.set, xscrollcommand=scrollbar_x.set)

    img_tk = ImageTk.PhotoImage(pil_img)

    canvas.create_image(0, 0, anchor="nw",
image=img_tk)

    canvas.image = img_tk


canvas.configure(scrollregion=canvas.bbox("a
ll"))

    root.mainloop()


def image_detection():

    file_path =
filedialog.askopenfilename(title="Select
Image", filetypes=[("Image Files",
".jpg;.jpeg;*.png")])

    if not file_path:

        return


    image = cv2.imread(file_path)

    if image is None:

        messagebox.showerror("Error", "Failed
to load the image.")

        return


    results1, results2 = detect_objects(image)

    result_image = draw_boxes(image, results1,
results2)

    result_image = scan_qr_code(result_image)

    show_image_in_window(result_image,
title="Detected Image")

def qr_code_from_gallery():

    file_path =
filedialog.askopenfilename(title="Select
Image", filetypes=[("Image Files",
".jpg;.jpeg;*.png")])
```

```
    if not file_path:
        return
    image = cv2.imread(file_path)
    if image is None:
        messagebox.showerror("Error", "Failed
to load the image.")
        return
    result_image = scan_qr_code(image)
    show_image_in_window(result_image,
title="QR Code Detection")
def real_time_detection(scan_qr=False):
    cap = cv2.VideoCapture(0)
  cap.set(cv2.CAP_PROP_FRAME_WIDTH,
640)
 cap.set(cv2.CAP_PROP_FRAME_HEIGHT,
480)
    cv2.namedWindow("Real-Time Detection",
cv2.WINDOW_NORMAL)
    cv2.resizeWindow("Real-Time Detection",
640, 480)
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        if scan_qr:
            frame = scan_qr_code(frame)
        else:
            results1, results2 =
detect_objects(frame)
            frame = draw_boxes(frame, results1,
results2)
            frame = scan_qr_code(frame)
        cv2.imshow("Real-Time Detection",
frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    cap.release()
    cv2.destroyAllWindows()
def create_gui():
    root = Tk()
    root.title("Object Detection & QR Code
Scanner")
    root.geometry("500x500")
    Label(root, text="Object Detection & QR
Code Scanner", font=("Arial", 16,
"bold")).pack(pady=10)
    Button(root, text="Real-Time Detection",
font=("Arial", 14), command=lambda:
real_time_detection(scan_qr=False)).pack(pad
y=10)
    Button(root, text="Real-Time QR Code
Scan", font=("Arial", 14), command=lambda:
real_time_detection(scan_qr=True)).pack(pad
y=10)
    Button(root, text="Image Object
Detection", font=("Arial", 14),
command=image_detection).pack(pady=10)
    Button(root, text="QR Code from Gallery",
font=("Arial", 14),
command=qr_code_from_gallery).pack(pady
=10)
    Button(root, text="Exit", font=("Arial", 14),
command=root.quit).pack(pady=10)
    root.mainloop()
if _name_ == "_main_":
    create_gui()
```

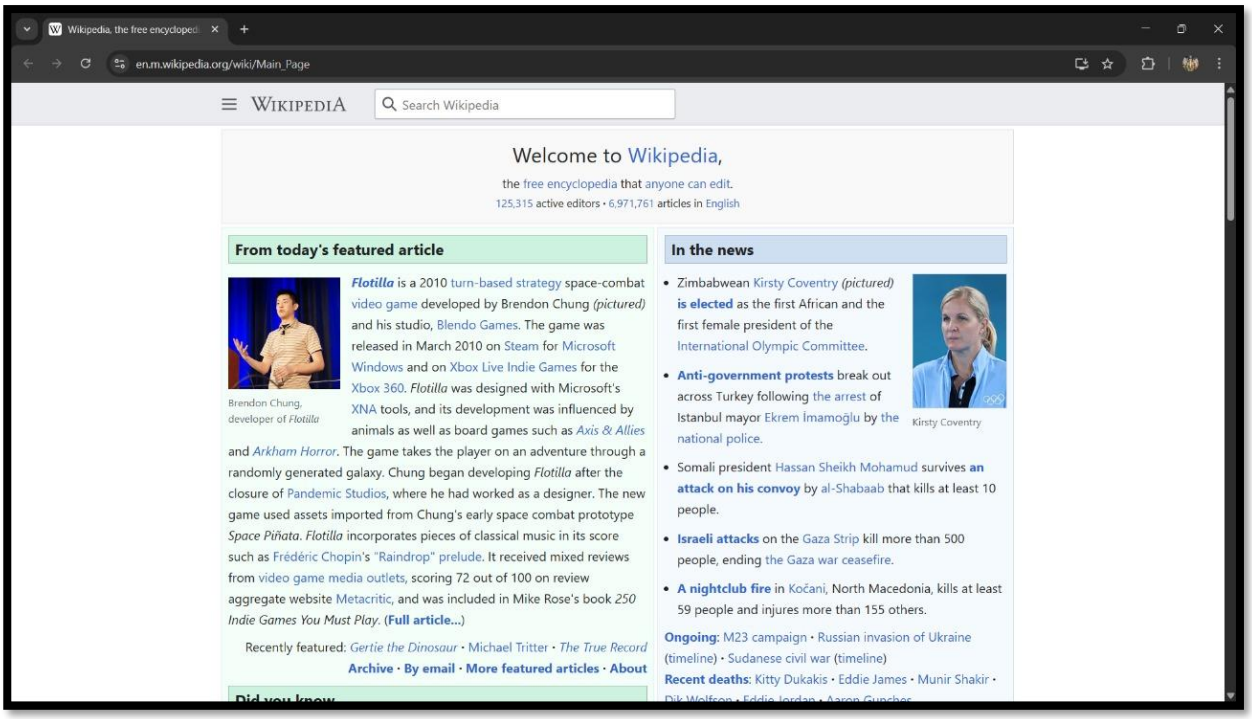
OUTPUT

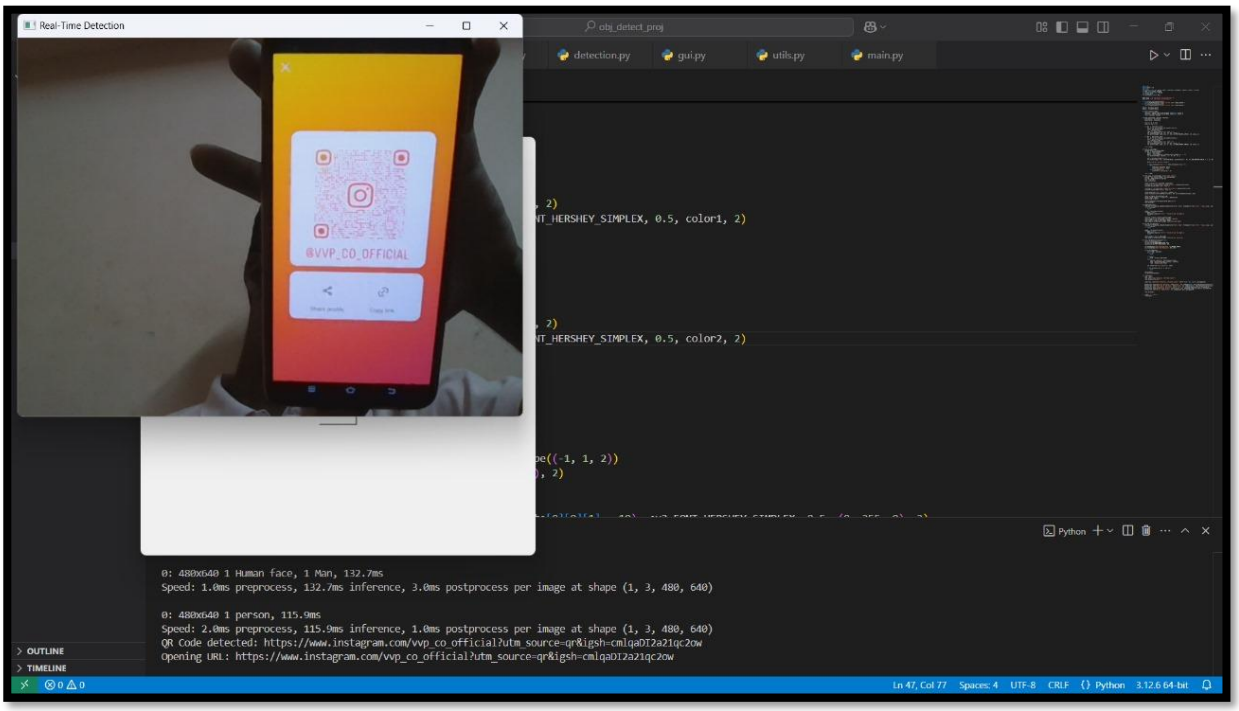

Object Detection & QR Code Scanner
Real-Time Detection
Real-Time QR Code Scan
Image Object Detection
QR Code from Gallery
Exit

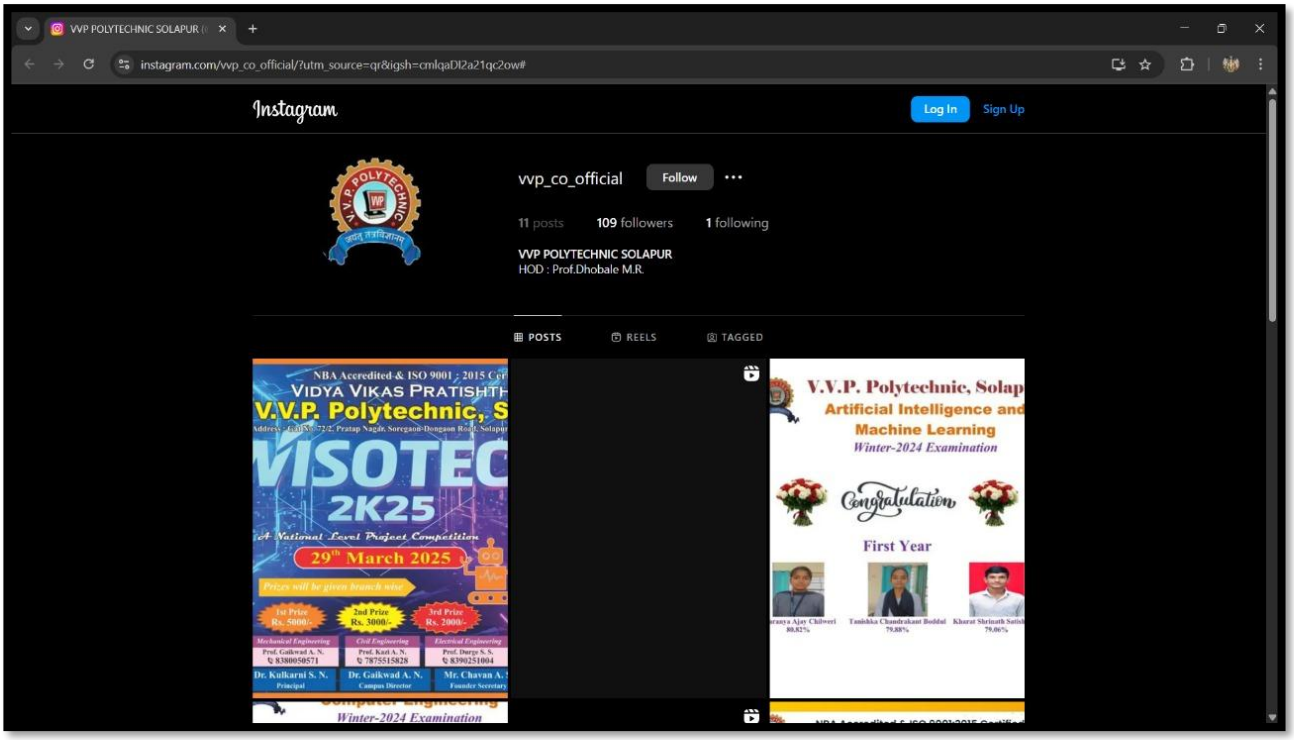

Real-Time Detection
person: 0.85
Man: 0.60
cell phone: 0.87
Human face: 0.74

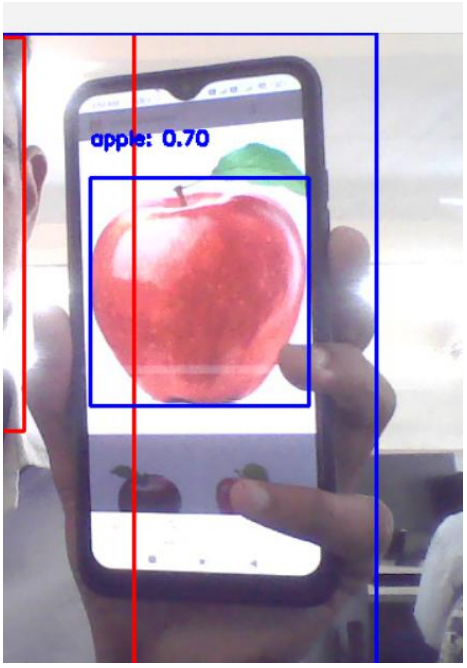

Detected Image
person: 0.89
person: 0.63
car: 0.64
person: 0.54
person: 0.61

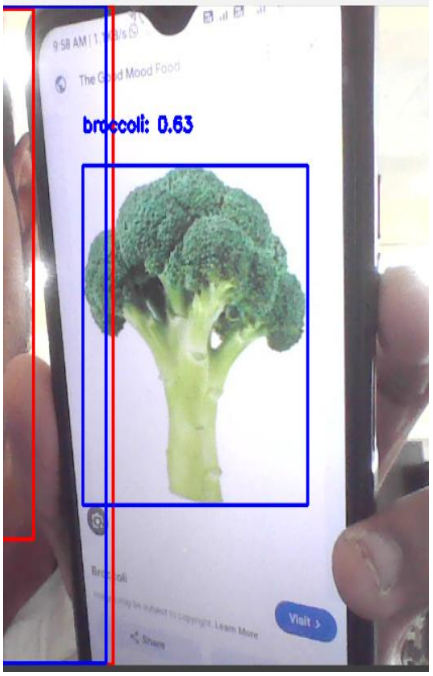
Detected Image
http://en.m.wikipedia.org

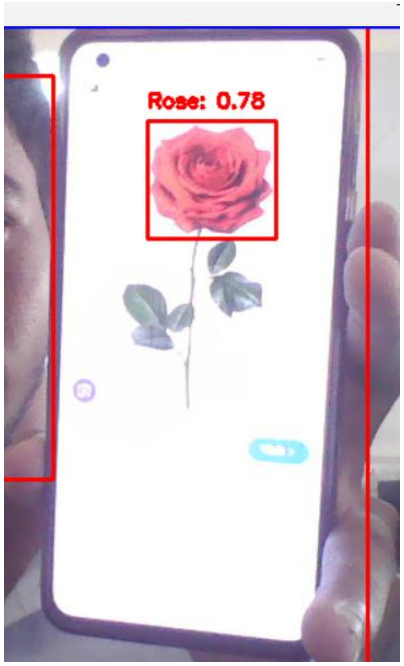

Wikipedia, the free encyclopedi
en.m.wikipedia.org/wiki/Main_Page
WIKIPEDIA
Search Wikipedia
Welcome to Wikipedia,
the free encyclopedia that anyone can edit.
125,315 active editors • 6,971,761 articles in English
From today's featured article
Flotilla is a 2010 turn-based strategy space-combat video game developed by Brendon Chung (pictured) and his studio, Blendo Games. The game was released in March 2010 on Steam for Microsoft Windows and on Xbox Live Indie Games for the Xbox 360. Flotilla was designed with Microsoft's XNA tools, and its development was influenced by animals as well as board games such as Axis & Allies and Arkham Horror. The game takes the player on an adventure through a randomly generated galaxy. Chung began developing Flotilla after the closure of Pandemic Studios, where he had worked as a designer. The new game used assets imported from Chung's early space combat prototype Space Piñata. Flotilla incorporates pieces of classical music in its score such as Frédéric Chopin's "Raindrop" prelude. It received mixed reviews from video game media outlets, scoring 72 out of 100 on review aggregate website Metacritic, and was included in Mike Rose's book 250 Indie Games You Must Play. (Full article...)
Brendon Chung, developer of Flotilla
Recently featured: Gertie the Dinosaur • Michael Tritter • The True Record
Archive • By email • More featured articles • About
In the news
Zimbabwean Kirsty Coventry (pictured) is elected as the first African and the first female president of the International Olympic Committee.
Anti-government protests break out across Turkey following the arrest of Istanbul mayor Ekrem İmamoğlu by the national police.
Somali president Hassan Sheikh Mohamud survives an attack on his convoy by al-Shabaab that kills at least 10 people.
Israeli attacks on the Gaza Strip kill more than 500 people, ending the Gaza war ceasefire.
A nightclub fire in Kočani, North Macedonia, kills at least 59 people and injures more than 155 others.
Kirsty Coventry
Ongoing: M23 campaign • Russian invasion of Ukraine (timeline) • Sudanese civil war (timeline)
Recent deaths: Kitty Dukakis • Eddie James • Munir Shakir •


Real-Time Detection
obj_detect_proj
detection.py
gui.py
utils.py
main.py
@VVP_CO_OFFICIAL
NT_HERSHEY_SIMPLEX, 0.5, color1, 2)
NT_HERSHEY_SIMPLEX, 0.5, color2, 2)
0: 480x640 1 Human face, 1 Man, 132.7ms
Speed: 1.0ms preprocess, 132.7ms inference, 3.0ms postprocess per image at shape (1, 3, 480, 640)
0: 480x640 1 person, 115.9ms
Speed: 2.0ms preprocess, 115.9ms inference, 1.0ms postprocess per image at shape (1, 3, 480, 640)
QR Code detected: https://www.instagram.com/vvp_co_official?utm_source=qr&igsh=cmlqaDI2a21qc2ow
Opening URL: https://www.instagram.com/vvp_co_official?utm_source=qr&igsh=cmlqaDI2a21qc2ow
OUTLINE
TIMELINE
Ln 47, Col 77
Spaces: 4
UTF-8
CRLF
Python
3.12.6 64-bit


VVP POLYTECHNIC SOLAPUR
instagram.com/vvp_co_official/?utm_source=qr&igsh=cmlqaDI2a21qc2ow#
Instagram
Log In
Sign Up
vvp_co_official
Follow
11 posts
109 followers
1 following
VVP POLYTECHNIC SOLAPUR
HOD : Prof.Dhobale M.R.
POSTS
REELS
TAGGED
VIDYA VIKAS PRATISHTH
V.V.P. Polytechnic, S
VISOTEC
2K25
29th March 2025
Winter-2024 Examination
V.V.P. Polytechnic, Solap
Artificial Intelligence and
Machine Learning
Winter-2024 Examination
Congratulation
First Year


apple: 0.70


broccoli: 0.63


Rose: 0.78

CHAPTER 7
TEST CASES

TESTCASES

1. Image Object Detection

Test Case ID	Test Scenario	Input	Expected Output
TC_Img_01	Load a valid image file	Valid image path	Image displayed with detected objects
TC_Img_02	Load an invalid image file	Corrupt image file	Error message: "Failed to load the image."
TC_Img_03	Load an image with no objects	Image with no objects	No bounding boxes drawn
TC_Img_04	Load an image with multiple objects	Image with multiple objects	Objects detected and labeled correctly
TC_Img_05	Cancel file selection	No file selected	No action performed
TC_Img_06	Select unsupported file format	Text file instead of image	Error message displayed

2. QR Code Detection from Gallery

Test Case ID	Test Scenario	Input	Expected Output
TC_QR_01	Load an image with a QR code	Image with QR code	QR code detected and displayed
TC_QR_02	Load an image without a QR code	Image without QR code	No QR code detected
TC_QR_03	Load a corrupted image	Corrupt image file	Error message: "Failed to load the image."
TC_QR_04	Cancel file selection	No file selected	No action performed
TC_QR_05	QR code with a URL	QR code containing URL	Web browser opens the URL
TC_QR_06	QR code with plain text	QR code with text	Text displayed on the image

3. Real-Time Object Detection

Test Case ID	Test Scenario	Input	Expected Output
TC_RT_01	Open real-time detection	Webcam stream	Live video feed with object detection
TC_RT_02	No objects in the frame	Empty background	No objects detected
TC_RT_03	Multiple objects in frame	Frame with multiple objects	Objects detected and labeled correctly
TC_RT_04	Webcam not available	No camera access	Error message displayed
TC_RT_05	Press 'q' to exit	Press 'q' key	Closes real-time detection window
TC_RT_06	Low lighting conditions	Dim environment	Objects detected but may have lower accuracy

4. Real-Time QR Code Scan

Test Case ID	Test Scenario	Input	Expected Output
TC_QRRT_01	QR code in camera frame	QR code in frame	QR code detected and displayed
TC_QRRT_02	No QR code in frame	No QR in view	No QR code detected
TC_QRRT_03	Blurry QR code	Blurry QR image	Detection may fail or be inaccurate
TC_QRRT_04	QR code with URL	QR code with URL	URL opened in web browser
TC_QRRT_05	Press 'q' to exit	Press 'q' key	Closes real-time QR detection window
TC_QRRT_06	Scanning multiple QR codes	Multiple QR codes in frame	All detected QR codes displayed

5.Functionality

Test Case ID	Test Scenario	Input	Expected Output
TC_GUI_01	Launch application	Run script	GUI opens successfully
TC_GUI_02	Click "Real-Time Detection"	Button click	Opens real-time object detection window
TC_GUI_03	Click "Real-Time QR Scan"	Button click	Opens real-time QR scan window
TC_GUI_04	Click "Image Object Detection"	Button click	Opens file selection dialog
TC_GUI_05	Click "QR Code from Gallery"	Button click	Opens file selection dialog
TC_GUI_06	Click "Exit"	Button click	Application closes

CHAPTER 8
CONCLUSION
& FUTURE SCOPE

CONCLUSION

Object detection using OpenCV helps us find and recognize objects in pictures or videos. It's a useful tool for many tasks like security cameras, self-driving cars, or mobile. OpenCV makes it simple to detect objects without needing to be an expert. With basic programming skills and the right tools, you can easily start building projects that use object detection.

FUTURE SCOPE

As technology improves, real-time object detection and image detection will become more advanced and useful. Here are some ways this technology can grow in the future:

1. Better AI Models
Future versions can use more advanced AI models like Vision Transformers (ViTs) and improved YOLO versions to make object detection more accurate and reliable.

2. Faster Processing
Using powerful hardware like GPUs and TPUs will help the system process images and detect objects quickly with less delay.

3. Improved Image Recognition
The system can be upgraded to recognize objects better, even in poor lighting, cluttered backgrounds, or different angles.

4. Support for Multiple Models
Users can choose from different detection models depending on their needs, improving flexibility and accuracy.

5. Offline Functionality
The system can be improved to work without an internet connection, making it more reliable in remote areas.

REFERENCES

1. REAL-TIME OBJECT RECOGNITION USING TENSORFLOW Aakanksha Singh[1] Madhavi Soni[2] Shreya Patkar[3] Asmita Deshmukh[4] 1Engineering Student, 2Engineering Student, 3Engineering Student, 4Assistant Professor 1Computer Engineering Department, 1K. C. College of Engineering and Management Studies and Research, Mumbai, India

2. Object Detection open cv using Python Agnes 1 , Sam salmon raja 2 , Pushparaj3 ,Abrar A4 2,3,4 Student(BTech),Department of Information Technology, Francis Xavier Engineering College, Tirunelveli, India 1 Assistant Professor(IT), Department of Information Technology, Francis Xavier Engineering College, Tirunelveli, Indi

3. Real Object Detection Using TensorFlow Milind Rane, Aseem Patil and Bhushan Barse. M. Rane · A. Patil (B) · B. Barse Department of Electronics Engineering, Vishwakarma Institute of Technology, Pune, India

4. A Review on Object Detection using Open CV Method Rajeshwar Kumar Dewangan1, Yamini Chouhan2 1,2Dept. of Computer Science & Engineering, Shri Shankar acharya Group of Institute, bhilai ,C.G., India

5. OBJECT DETECTION USING TENSOR FLOW OPENCV AND FLASK Hemalatha T*1, K Venkata Chalapathi*2 *1,2Asisstant Professor, Department of CSE, S V U College of Engineering, Tirupati, AP, India.

ACTION PLAN

Stage 1: Analysis, Literature Survey, Forming problem statement	1-Aug-2024 to 24-Sep-2024
Stage 2: Designing	1-Oct-2024 to 30-Oct -2024
Stage 3: Implementation	1-Nov -2024 to 30-Jan-2025
Stage 4: Integration of Modules & Testing	1-Feb-2025 to 28-Feb -2025
Stage 5: Deployment & Results	1-Mar-2025 to 15-Mar-2025

CERTIFICATE

NBA Accredited

ISO 9001-2015 certified

V.V.P. POLYTECHNIC

जयतु तंत्रविज्ञानम्

Vidya Vikas Pratisthan's

V.V.P. Polytechnic, Solapur

WISOTECH 2k25

A National Level Project Competition

Event Date SATURDAY 29th MARCH

Certificate

This Certificate is awarded to

Mr. / Miss. Shivdatt Vanmane.

of Computer / Civil / Electrical / Mechanical / ENTC

of V.V.P Polytechnic, Solapur.

for Participating / Winning Participating.

in A National Level Project Competition, Wisotech-2K25 held on 29-03-2025.

Prof. Biradar R. Y.
Co-ordinator

Dr. Kulkarni S. N.
Principal